

DevAgent 智能体系统设计方案

系统架构设计说明 — 模块划分 · 数据流 · 核心接口 · 技术选型

开发组织：DevAgent 开发团队

DevAgent 智能体系统设计方案

系统架构设计说明 — 模块划分 · 数据流 · 核心接口 · 技术选型

开发组织：DevAgent 开发团队

日期：2026-06-04

1. 系统架构概览

1.1 分层架构

DevAgent 采用四层分层架构设计：

1.2 系统架构图



	LLMClient	ToolRegistry	SandboxMgr	ContextMgr	
	(双模型)	(30 tools)	(Docker)	(4层上下)	

1.3 核心数据流

全流程开发 (--mode full) 的数据流：



2. 模块划分与职责

2.1 核心模块 (agent_core)

2.2 Agentic 引擎模块 (agentic)

2.3 专业 Agent 模块 (agents)

3. 核心类/接口设计

3.1 AgentState — 统一状态对象

```

@dataclass
class AgentState:
    task_id: str      # UUID 任务标识
    task_type: str    # design / implement / repair / full
    input_path: str   # 输入文件路径
    output_root: str  # 产物输出根目录
    language: str     # python / java

    # 各阶段产出
    requirements: dict # 结构化需求分析结果
    design_artifacts: dict # 设计文档与图表
    code_files: list[str] # 生成的源码文件路径列表
    test_files: list[str] # 测试文件路径列表
  
```

```

test_results: dict    # pytest 执行结果 + 覆盖率
debug_analysis: dict  # 错误定位与根因分析
repair_patch: dict    # 修复补丁 + 修改文件列表

# 控制字段
retry_count: int      # 当前重试次数
max_retry: int        # 最大重试次数
status: str           #  workflow 状态机

def add_trace(node, status, details) # 执行追踪
def add_error(phase, message)        # 错误记录
def get_output_subdir(phase) -> str  # 阶段产物目录

```

3.2 BaseTool — 工具抽象接口

```

class BaseTool(ABC):
    name: str          # 工具名称 (LLM function calling 用)
    description: str    # 工具描述 (LLM 决策依据)
    parameters: dict    # JSON Schema 参数定义

    @abstractmethod
    async def execute(params: dict, workspace: str) -> ToolResult

    def to_openai_schema() -> dict # 生成 OpenAI Function Calling Schema
    def description_text() -> str  # 人类可读的工具说明

```

3.3 WorkflowController — Pipeline 编排器

```

class WorkflowController:
    def __init__(config_path)    # 加载配置 + 初始化所有 Agent
    def execute(spec: TaskSpec) -> AgentState # 主入口

    #  workflow 实现
    def _run_full_workflow(state) # Plan→Reqs→Design→Code→Test→Debug/Repair
    def _run_design_workflow(state) # Plan→Reqs→Design
    def _run_implement_workflow(state) # Plan→Reqs→Code→Test
    def _run_test_workflow(state) # 独立测试执行
    def _run_debug_workflow(state) # 独立调试分析
    def _run_repair_workflow(state) # 独立修复 + 回归
    def _run_agentic_workflow(state) # 委托给 DevAgentCore

    # 编排控制
    def _execute_phase(state, phase, agent_fn) -> PhaseResult # 阶段执行 + 质量门
    def _retry_phase(state, phase, agent_fn, max_retry) -> bool # 上下文保留重试
    def _run_repair_loop(state) # 修复+回归循环 (最多 max_retry 轮)
    def _run_testing_with_sandbox(state) # 沙箱隔离测试执行

```

4. 关键技术选型与理由

5. 可扩展性设计

5.1 新增语言支持

通过以下步骤添加新编程语言（如 Go、TypeScript）：

在 CodeAgent 中编写 LANGUAGE_CODE_PROMPT（项目结构、测试框架、安全规则、质量检查清单）

在 AgentState.language 中添加语言标识符

在 TestAgent 中添加对应测试框架的执行器（如 go test、jest）

在 StaticAnalyzer 中添加语言的 AST 分析规则

5.2 新增图表类型

diagrams.py 中的图表生成函数遵循统一模式：输入结构化 dict → 输出 Mermaid/PlantUML 文本。添加新图表类型只需：

在 diagrams.py 中实现 new_diagram(params: dict) -> str 函数

在 DesignAgent 的 JSON Schema 中添加对应字段

在 renderer.py 的 render_from_design() 中添加渲染调用

5.3 插件体系

VSCode/IntelliJ/Eclipse 三端 IDE 插件均通过 WebSocket 连接到 DevAgent 后端 API，实现统一的双向通信协议：

```
WebSocket 事件协议：
→ review.requested # Agent 请求阶段审核
← review.response # 用户响应（批准/修改/拒绝）
→ agent.question # Agent 向用户提问
← agent.answer # 用户回答
→ approval.requested # 危险操作审批请求
← approval.response # 用户审批决策
→ progress.snapshot # 实时进度快照
← command.pause # 用户暂停
← command.resume # 用户恢复
```

层次	模块	职责
表现层	CLI (cli/main.py) + REST API (api/app.py) + VS	用户交互入口，任务提交，实时审核
编排层	WorkflowController + DevAgentCore + Planner	任务编排和执行调度
专业层	8 个 V1 Agent + Multi-Agent 协作引擎	需求分析、设计、编码、测试、调试、修复、审核
基础设施层	LLMClient + ToolRegistry + SandboxManager +	模型调用、工具执行、安全隔离、上下文管理

模块文件	核心类/函数	职责
workflow.py	WorkflowController	Pipeline 编排：Plan→Reqs→Design→Code→Test→De
state.py	AgentState	统一状态对象：贯穿整个工作流，携带各阶段产出和错误信
llm_client.py	LLMClient	双模型统一接口：chat_structured() + chat_stream()，
config_loader.py	load_config/get_llm_config	YAML 配置加载，模型参数（temperature/top_p/max_
schemas.py	TaskSpec/Artifact	任务规格验证（Pydantic）和产物元数据模型
langgraph_workflow.py	LangGraphWorkflow	可选的 LangGraph 状态图工作流引擎
router.py	Router	任务类型路由：根据 mode 参数分发到对应工作流

模块文件	核心类/函数	职责
core.py	DevAgentCore	ReAct 主循环：Think→Act→Observe，工具调度，终止
tools.py	ToolRegistry/BaseTool	工具注册、Schema 生成、异步执行派发
context.py	ContextManager	四层上下文管理 + 阶段检测 + 幻觉防治
fault_locator.py	FaultLocalizationPipeline	三层故障定位：SBFL + Static + LLM Fusion
planning.py	PlannerAgent/PlanExecutor	任务分解为 SubTask DAG，拓扑排序调度
multi_agent.py	Coordinator/WorkerAgent	多 Agent 协作：文件级锁 + 冲突解决
sandbox.py	SandboxManager	Docker/Podman/Local 三级沙箱隔离执行
pipeline_tools.py	_AgentToolAdapter	V1 Agent → V2 工具适配器
interaction.py	InteractionController	人机交互：审批流、暂停/恢复、进度推送
experience.py	ExperienceStore	经验积累：成功/失败模式记录与检索
validation.py	InstantValidator	工具调用即时验证（路径安全、参数类型）

observability.py	StreamingServer	SSE 事件流 + 任务历史持久化
------------------	-----------------	-------------------

Agent	核心职责	输入	输出
PlannerAgent	任务分解	任务描述 + 仓库映射	ExecutionPlan (SubTask DAG)
RequirementAgent	需求分析	requirements.md	结构化需求 JSON + SRS 文档
DesignAgent	架构设计	需求 JSON	设计 JSON + SDD 文档 + 图表 PNG
CodeAgent	代码生成	需求/设计 JSON	Python/Java 项目 + README
TestAgent	测试生成与执行	代码文件	pytest 测试套件 + 覆盖率报告
DebugAgent	根因分析	测试失败信息	Bug 分类 + 定位 + 修复假设
RepairAgent	代码修复	调试分析 + 源码	patch.diff + 回归测试结果
ReviewAgent	报告生成	所有阶段产出	执行摘要 + 质量仪表盘

技术决策	选型	理由
编程语言	Python 3.11+	丰富的 LLM/AI 生态 (openai, langchain) ；工程级类型系统
LLM API 协议	OpenAI-compatible	业界事实标准；DeepSeek 和大多数国产模型均提供兼容接口
代码生成目标	Python（主）+ Java（辅）	Python 在 AI/DS 领域占主导，Java 在企业级应用中广泛使用
测试框架	pytest + coverage.py	Python 生态最成熟的测试框架；coverage.py 提供精确覆盖率
图表渲染	kroki.io (Mermaid+PlantUML→PNG)	无需本地安装；同时支持 Mermaid 和 PlantUML 两种图表
沙箱隔离	Docker → Podman → Local	三层自动回退；network=None 确保网络安全；资源限制
Web 框架	FastAPI + WebSocket	异步高性能；原生 WebSocket 支持双向实时通信
IDE 集成	VSCode + IntelliJ + Eclipse	覆盖三大主流 IDE；通过 WebSocket 实现实时双向交互
配置管理	YAML + pydantic-settings	YAML 可读性高；pydantic 提供运行时类型验证
文档输出	Markdown + IEEE 标准模板	Markdown 可渲染为 PDF/HTML/Word；IEEE 830/1016/1017 标准
故障定位	SBFL(Ochiai) + AST 静态分析 + LLM	三重互补：SBFL 提供统计证据，静态分析提供规则检查，LLM 提供推理